PYTHON - FUNÇÕES, MODULARIZAÇÃO E POO

Sequência didática - Aula 01 a Aula 20

O material inicia com funções, avança para modularização e Programação Orientada a Objetos e termina com a aplicação dos conceitos em um sistema de agendamento.

AULA 01 - INTRODUÇÃO ÀS FUNÇÕES COM def

Uma função é um bloco de código reutilizável que executa uma tarefa específica. A palavra def é utilizada para definir uma função.

```
def mensagem():
    print("Olá")

mensagem()
```

AULA 02 - PARÂMETROS E return

Parâmetros são valores recebidos pela função. O return devolve um resultado para quem chamou a função.

```
def somar(a, b):
    return a + b

resultado = somar(10, 5)
print(resultado)
```

AULA 03 - FUNÇÕES NA PRÁTICA - CALCULADORA

Nesta aula, diferentes operações são organizadas em funções.

```
def somar(a, b):
    return a + b

def subtrair(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

def dividir(a, b):
    if b == 0:
        return "Erro: Divisão por zero!"
    return a / b

print(somar(10, 5))
print(subtrair(10, 5))
print(multiplicar(10, 5))
print(dividir(10, 2))
```

AULA 04 - MODULARIZAÇÃO - SEPARANDO O CÓDIGO

Modularização permite dividir o programa em arquivos menores. Um arquivo pode guardar funções e outro pode executar o programa.

```
meu_projeto/
|
|-- calculadora.py
`-- main.py
```

```
# calculadora.py
def somar(a, b):
    return a + b

def subtrair(a, b):
    return a - b
```

AULA 05 - IMPORT E CHAMADA DE FUNÇÕES

O comando import permite utilizar funções definidas em outro arquivo.

```
import calculadora

n1 = float(input("Digite o primeiro número: "))
n2 = float(input("Digite o segundo número: "))

resultado = calculadora.somar(n1, n2)
print(resultado)
```

Também é possível importar funções específicas:

```
from calculadora import somar, multiplicar

print(somar(10, 5))
```

AULA 06 - INTRODUÇÃO À POO

POO significa Programação Orientada a Objetos. É um paradigma utilizado para organizar melhor os códigos utilizando objetos e classes. Objetos possuem atributos (características) e métodos (ações).

AULA 07 - CLASSES E OBJETOS

Uma classe é um modelo utilizado para criar objetos. A palavra reservada para criar classes é class.

```
class Carro:
    marca = "Toyota"
    modelo = "Corolla"

carro1 = Carro()

print(carro1.marca)
print(carro1.modelo)
```

AULA 08 - MÉTODO CONSTRUTOR __init__

O método __init__() é o construtor da classe. Ele é executado automaticamente quando um objeto é criado e é utilizado para inicializar atributos.

```
class Pessoa:

    def __init__(self, nome):
        self.nome = nome

pessoa1 = Pessoa("Carlos")
print(pessoa1.nome)
```

AULA 09 - ATRIBUTOS E PARÂMETROS EM CLASSES

Os parâmetros permitem enviar dados para o objeto no momento de sua criação. Esses valores podem ser armazenados nos atributos utilizando self.

```
class Aluno:

    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

aluno1 = Aluno("Maria", 20)

print(aluno1.nome)
print(aluno1.idade)
```

AULA 10 - MÉTODOS DE OBJETOS

Métodos são funções criadas dentro de uma classe. Eles representam ações que um objeto pode executar.

```
class Saudacao:

    def mensagem(self, nome):
        print(f"Olá {nome}")

usuario = Saudacao()
usuario.mensagem("Ana")
```

AULA 11 - PROPRIEDADE DE CLASSE

Uma propriedade de classe pertence à classe inteira e pode ser compartilhada pelos objetos.

```
class Produto:
    imposto = 0.10

produto1 = Produto()
produto2 = Produto()

print(produto1.imposto)
```

```
print(produto2.imposto)
```

AULA 12 - MÉTODO DE CLASSE

Métodos de classe utilizam o decorador @classmethod e trabalham diretamente com a classe.

```
class Curso:
    categoria = "Programação"

    @classmethod
    def exibir_categoria(cls):
        print(cls.categoria)

Curso.exibir_categoria()
```

AULA 13 - HERANÇA

Herança permite que uma classe herde características e métodos de outra classe.

```
class Animal:

    def falar(self):
        print("O animal faz som")

class Cachorro(Animal):
    pass

dog = Cachorro()
dog.falar()
```

AULA 14 - POLIMORFISMO

Polimorfismo permite que métodos com o mesmo nome apresentem comportamentos diferentes.

```
class Ave:

    def som(self):
        print("Som da ave")

class Pato(Ave):

    def som(self):
        print("Quack")

pato = Pato()
pato.som()
```

AULA 15 - ENCAPSULAMENTO

Encapsulamento é utilizado para proteger atributos da classe. Neste exemplo é utilizado __ antes do atributo.

```
class Usuario:

    def __init__(self):
        self.__senha = "123456"

usuario = Usuario()
```

AULA 16 - CLASSES INTERNAS

Classes internas são classes criadas dentro de outras classes.

```
class Universidade:

    class Curso:

        def mensagem(self):
            print("Curso de Python")

curso1 = Universidade.Curso()
curso1.mensagem()
```

AULA 17 - ORGANIZAÇÃO DE UM PROJETO POO EM ARQUIVOS

Em projetos maiores, as responsabilidades podem ser distribuídas em arquivos diferentes. As classes ficam separadas do arquivo responsável pelo menu e das funções do sistema.

```
projeto/
|
|-- telamenu.py
|-- sistema.py
|-- classservico.py
`-- classagendamento.py
```

telamenu.py: apresenta as opções ao usuário.

sistema.py: contém as funções chamadas pelo menu.

classservico.py: contém a classe Servico.

classagendamento.py: contém a classe Agendamento.

AULA 18 - MENU COM while E match case

O while permite manter o menu em execução. O match case verifica a opção escolhida pelo usuário.

```
while True:

    print("""
1 - Cadastrar serviço
2 - Listar serviço
3 - Agendar serviço
4 - Ver agendamento
5 - Sair
""")

    opcao = input("Escolha: ")

    match opcao:

        case "1":
            print("Cadastrar serviço")

        case "2":
            print("Listar serviço")

        case "3":
            print("Agendar serviço")

        case "4":
            print("Ver agendamento")

        case "5":
            break
```

AULA 19 - INTEGRAÇÃO: MENU, FUNÇÃO E CLASSE

Nesta etapa, o aluno observa o fluxo entre os arquivos. O menu chama uma função do sistema; a função utiliza uma classe para criar o objeto.

```
telamenu.py
     |
     |  sistema.cadastrar_servico()
     v
sistema.py
     |
     |  Servico(nome, valor, data, horario)
     v
classservico.py
```

```
# telamenu.py
import sistema

case "1":
    sistema.cadastrar_servico()
```

```
# sistema.py
from classservico import Servico

def cadastrar_servico():
    nome = input("Nome do serviço: ")
    valor = float(input("Valor: "))
    data = input("Data: ")
    horario = input("Horário: ")

    servico = Servico(nome, valor, data, horario)
```

AULA 20 - ATIVIDADE PRÁTICA DE POO

Antes do projeto final, desenvolva atividades para revisar classes, objetos, métodos e organização do código.

1. Crie uma classe Aluno com nome e idade e mostre os dados.

2. Crie uma classe Calculadora com método de soma.

3. Crie uma herança utilizando as classes Animal e Gato.

4. Crie um exemplo de encapsulamento utilizando uma senha privada.

5. Crie uma classe interna chamada Motor dentro da classe Carro.

6. Crie dois arquivos Python: um contendo uma classe e outro responsável por importar e criar um objeto dessa classe.

SISTEMA DE AGENDAMENTO PARA SALÃO DE BELEZA

SITUAÇÃO-PROBLEMA

Um salão de beleza realiza diariamente serviços como corte de cabelo, manicure, escova e outros atendimentos. Atualmente, o controle dos serviços e agendamentos é realizado de maneira pouco organizada, dificultando a consulta das informações dos clientes e dos horários reservados.

O salão solicitou o desenvolvimento de um sistema em Python que permita cadastrar os serviços oferecidos e realizar agendamentos para os clientes.

O sistema deverá ser desenvolvido utilizando conceitos básicos de Programação Orientada a Objetos (POO) e organizado em arquivos separados, de modo que cada parte do programa possua uma responsabilidade específica.

DESAFIO

Você foi contratado para desenvolver o sistema de agendamento do salão. A aplicação deverá funcionar através de um menu no terminal, permitindo ao usuário escolher as operações disponíveis.

O sistema deverá permitir cadastrar um serviço, informando nome, valor, data e horário; listar o serviço cadastrado; realizar um agendamento, associando o nome de um cliente ao serviço; visualizar os dados do agendamento; e encerrar o programa.

ESTRUTURA DO PROJETO

```
projeto/
|
|-- telamenu.py
|-- sistema.py
|-- classservico.py
`-- classagendamento.py
```

MENU ESPERADO

```
1 - Cadastrar serviço
2 - Listar serviço
3 - Agendar serviço
4 - Ver agendamento
5 - Sair
```

RESULTADO ESPERADO

Ao final da atividade, o projeto deverá demonstrar que o aluno consegue criar classes e objetos, utilizar métodos e funções, importar classes e arquivos e organizar um programa Python em diferentes arquivos de acordo com suas responsabilidades.